UNIVERSITAT DE LES ILLES BALEARS

Diseño e Implementación de un Compilador

Compiladores - 21780

Carlos Garrido del Toro
41659365V

2025-2026

Índice

1. Introducción **2**

2. Diseño del Lenguaje **3**
2.1. SINTAXIS . . . 3
2.2. SEMÁNTICA . . . 6

3. Estructuras Globales **8**
3.1. Tabla de Símbolos . . . 8

4. Front-End (Análisis) **10**
4.1. Análisis Léxico (Scanner) con JFlex . . . 10
4.2. Análisis Sintáctico (Parser) . . . 11
4.3. Representación Intermedia: Árbol Sintáctico Abstracto (AST) . . . 12
4.4. Análisis Semántico . . . 15
4.5. Implementación del Analizador Semántico . . . 16

5. Back-End (Síntesis) **18**
5.1. Generación de Código Intermedio . . . 18
5.2. Implementación del Generador de Código Intermedio . . . 22
5.3. Generación de Código Ensamblador . . . 25
5.4. Implementación del Generador de Ensamblador . . . 25

6. Guía de Usuario **28**
6.1. Requisitos Previos . . . 28
6.2. Lenguaje . . . 28
6.3. Compilación y Ejecución . . . 30

7. Casos de Prueba y Resultados **31**
7.1. Caso Correcto 1: Sistema de Tipos y Estructuras de Datos . . . 31
7.2. Caso Correcto 2: Control de Flujo y Funciones . . . 32
7.3. Caso Correcto 3: Expresiones Complejas y Operadores . . . 33
7.4. Caso Correcto 4: Evaluación de Expresiones Aritmético-Lógicas . . . 34
7.5. Caso Correcto 5: Entrada/Salida e Interactividad (Calculadora) . . . 34
7.6. Caso Erróneo 1: Tipos y Semántica . . . 35
7.7. Caso Erróneo 2: Recuperación de Errores Sintácticos y Semánticos . . . 36
7.8. Caso Erróneo 3: Recuperación sintáctica de la Calculadora . . . 37
7.9. Presentación de la práctica . . . 38

8. Conclusión **38**

Mejoras implementadas en la entrega extraordinaria

A partir de los comentarios de la anterior entrega, se han implementado las siguientes mejoras en el (`.sh` y `.bat`):

1. **Entorno Limpio:** Se ha optimizado la gestión de la carpeta de salida `ArchivoCompilacion` para evitar archivos de antiguas ejecuciones.
   - **Antes:** Los archivos se sobrescribían, lo que permitía que informes de errores antiguos (`0_errores.txt`) o binarios desactualizados persistieran tras una compilación fallida, generando información falsa en nuevas ejecuciones.
   - **Ahora:** El script asegura una limpieza total de la carpeta antes de cada ejecución, garantizando que los resultados correspondan al estado actual del código.
2. **Compatibilidad (GCC 32/64-bit):** Se ha corregido el error de enlazado en Windows (`.bat`) al utilizar distribuciones de GCC de 64 bits.
   - Se ha forzado el uso del flag `-m32` y se han gestionado los símbolos de entrada mediante `-defsym`. Esto asegura que el código ensamblador generado (x86 legado) sea compatible con compiladores modernos de 64 bits.
3. **Optimización de la Interfaz de Consola:** Se han desactivado los mensajes de depuración con el objetivo de proporcionar una salida más limpia, donde el usuario final solo reciba la información relevante del programa ejecutado y los resultados del cálculo.
4. **Actualización de la documentación:** Se han añadido nuevas aclaraciones de configuración en la *Guía de Usuario*.

1 Introducción

Esta práctica se basa en la creación de un compilador desde cero, abarcando todas las fases del proceso: análisis léxico, sintáctico, construcción del Árbol Sintáctico Abstracto (AST), análisis semántico, generación de código intermedio y construcción del ejecutable final.

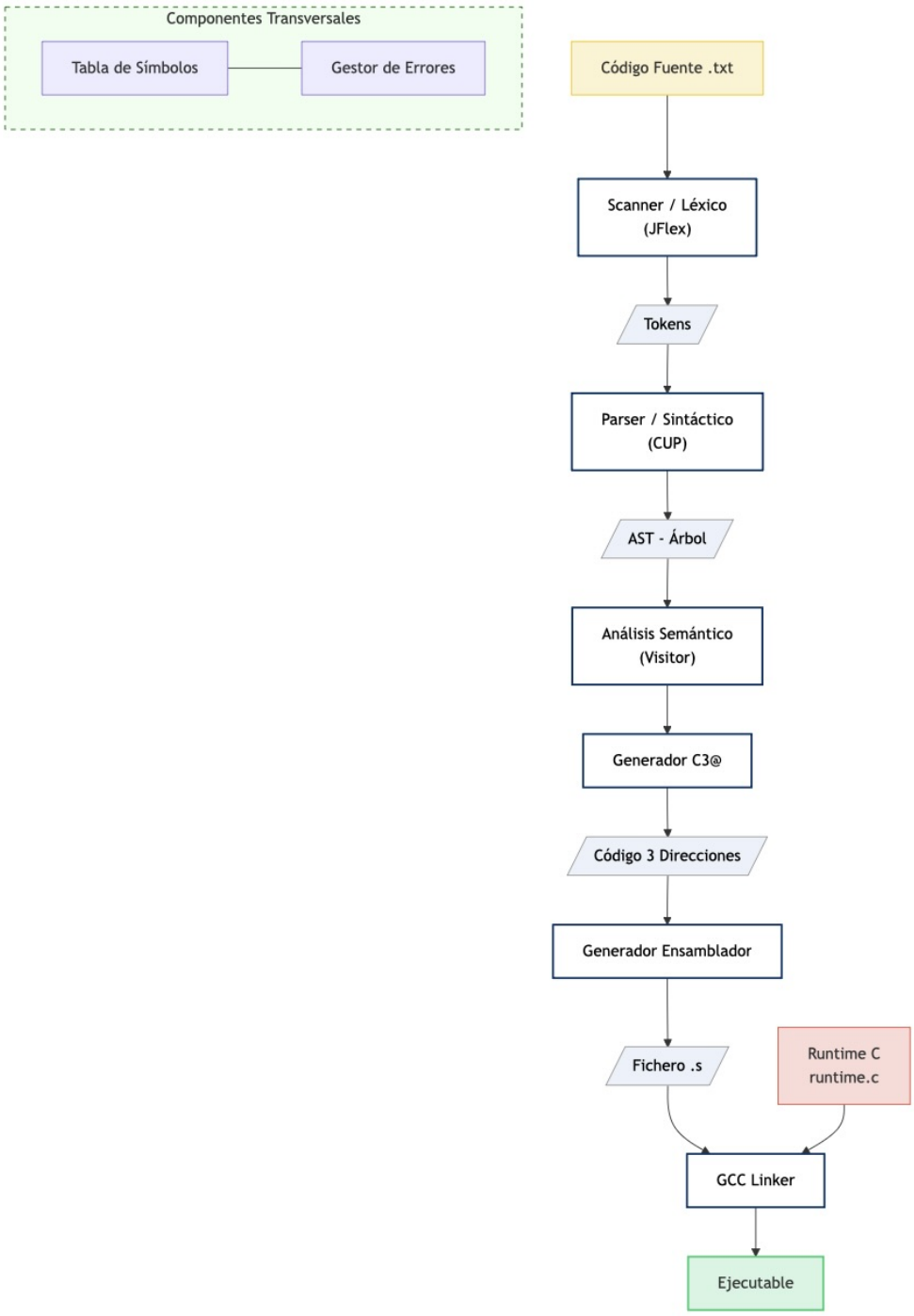

Figura 1: Arquitectura global del compilador: Flujo de datos desde el código fuente hasta el ejecutable.

2 Diseño del Lenguaje

2.1 SINTAXIS

2.1.1 Léxicos (TOKENS)

En un primer paso, declararemos la sintaxis de nuestro programa, los tokens que este detectará:

Categoría	Definición
Identificadores	→ [A-Za-z_][A-Za-z0-9_]*
Tipos	→ `int` | `char` | `bool`
Literales	→ Entero | Caracter | Logico
Entero	→ `0` | [1-9][0-9]*
Caracter	→ `'`(CaracterSimple | SecuenciaEscape)`'`
CaracterSimple	→ [`^'\n`]
SecuenciaEscape	→ \(n | t | r | \\ | ' | ")
Logico	→ `true` | `false`
Op. Aritméticos	→ `+` | `-` | `*` | `/` | `%`
Op. Relacionales	→ `==` | `!=` | `<` | `>` | `<=` | `>=`
Op. Lógicos	→ `&&` | `||` | `!`
Asignación	→ `=`
Delimitadores	→ `(` | `)` | `{` | `}` | `[` | `]` | `;` | `,` | `:`

2.1.2 Sintaxis: Estructura Global

Producción	Definición
Programa	→ DeclaracionGlobal* CuerpoPrincipal
DeclaracionGlobal	→ DeclaracionSubprograma | DeclaracionVar
CuerpoPrincipal	→ `main` { DeclaracionVar* Sentencia+ }
DeclaracionSubprograma	→ Tipo Funcion | `void` Procedimiento
Funcion	→ Id (Parametros?) { DeclaracionVar* Sentencia+ }
Procedimiento	→ Id (Parametros?) { DeclaracionVar* Sentencia+ }
Parametros	→ Parametro (, Parametro)*
Parametro	→ Tipo Id

2.1.3 Declaración de Variables

Producción	Definición
DeclaracionVar	→ Tipo ListaId ;
ListaId	→ ItemDeclaracion (, ItemDeclaracion)*
ItemDeclaracion	→ Id ArrayDecl?
ArrayDecl	→ [Entero]

2.1.4 Sentencias

Sentencia	Definición
Sentencia	→ Declaracion | Asignacion | Condicional | Switch | Bucles | LlamadaSentencia | Return | E_S
Declaracion	→ Tipo Id (= Expresion)? (, Id ...)* ;
Asignacion	→ LValue = Expresion ;
LValue	→ Id ([Expresion])?
Condicional	→ `if` (Expresion) { Sentencia* } (`else` { Sentencia* })?
Switch	→ `switch` (LValue) { Caso+ CasoDefault? }
Caso	→ `case` Literal : Sentencia* `break` ;
CasoDefault	→ `default` : Sentencia*
Bucles	→ WhileLoop | ForLoop
WhileLoop	→ `while` (Expresion) { Sentencia* }
ForLoop	→ `for` (Asignacion? ; Expresion ; Asignacion?) { Sentencia* }
LlamadaSentencia	→ LlamadaFunciones ;
Return	→ `return` Expresion? ;
E_S	→ Input | Output
Input	→ `leer` (LValue) ;
Output	→ `imprimir` (Expresion) ;

2.1.5 Expresiones Lógicas y Aritméticas

Expresión	Definición
Expresion	→ ExprRelacional ExprLogica_cont
ExprLogica_cont	→ (`&&` | `||`) ExprRelacional ExprLogica_cont | λ
ExprRelacional	→ ExprAritmetica ExprRel_cont
ExprRel_cont	→ OP_RELACIONAL ExprAritmetica ExprRel_cont | λ
ExprAritmetica	→ Termino ExprArit_cont
ExprArit_cont	→ (`+` | `-`) Termino ExprArit_cont | λ
Termino	→ Factor Termino_cont
Termino_cont	→ (`*` | `/` | `%`) Factor Termino_cont | λ
Factor	→ (Expresion) | `!` Factor | `-` Factor | LValue | Literal | LlamadaFunciones
LlamadaFunciones	→ Id ((Expresion (, Expresion)*)?)

2.1.6 Precedencia y Asociatividad de Operadores

Para resolver ambigüedades en la evaluación de expresiones, el lenguaje define los siguientes niveles de precedencia (de mayor a menor) y su asociatividad:

Nivel	Operadores	Asociatividad
1 (Más alta)	( ) [ ]	Izquierda a Derecha
2	- (unario) !	Derecha a Izquierda
3	* / %	Izquierda a Derecha
4	+ -	Izquierda a Derecha
5	< > <= >=	Izquierda a Derecha
6	== !=	Izquierda a Derecha
7	&&	Izquierda a Derecha
8 (Más baja)	||	Izquierda a Derecha

2.2 SEMÁNTICA

Se definen las reglas de compatibilidad de tipos implementadas. El compilador sigue un enfoque estricto para garantizar la seguridad de tipos y la integridad de la memoria.

2.2.1 Operaciones Aritméticas (+, -, *, /, %)

Op 1	Op 2	Res.	Nota
int / char	int / char	int	Los caracteres se tratan como su valor numérico ASCII (enteros de 32 bits), permitiendo aritmética mixta.

Nota sobre el Operador Módulo (%)

El operador de módulo es exclusivo para tipos enteros.

2.2.2 Operaciones Unarias (!, -)

Op.	Operando	Res.	Nota
!	bool	bool	Negación lógica
-	int	int	Cambio de signo (Menos unario)

2.2.3 Operaciones Relacionales (<, >, <=, >=, ==, !=)

Op 1	Op 2	Res.	Nota
int	int	bool	Comparación numérica
bool	bool	bool	Solo igualdad (==) y desigualdad (!=)

2.2.4 Operaciones Lógicas (&&, ||)

Op 1	Op 2	Res.	Nota
bool	bool	bool	Lógica booleana estándar

2.2.5 Reglas de Asignación (=)

LValue	Valor	Acción
int	int	Asignación directa
int	char	Asignación directa (ASCII a Entero)
char	char	Asignación directa
char	int	Asignación directa (Truncamiento implícito, se trata como entero)
bool	bool	Asignación directa

2.2.6 Entrada y Salida

Las sentencias de comunicación con el exterior soportan tipos básicos:

- **leer(var):** Admite variables de tipo `int` y `char`. El compilador selecciona automáticamente la rutina de lectura adecuada según el tipo de la variable.
- **imprimir(exp):** Admite expresiones de tipo `int`, `char` y `bool`.

2.2.7 Arrays (Vectores)

El compilador soporta arrays unidimensionales de tipos primitivos (`int` y `char`).
Declaración:
El tamaño del array debe ser definido obligatoriamente mediante un literal entero positivo (ej. `int/char v[10];`).

Acceso e Indexación:
Para acceder a una posición (`v[i]`), el índice `i` siempre debe ser de tipo `int`.

Base	Índice	Resultado
Array	`int`	Tipo base del array (`int` o `char`)
Array	`char` / `bool`	**Error**: el índice debe ser entero
No array	`int`	**Error**: variable no indexable

Asignación a Array:
En una instrucción del tipo `v[i] = valor`, se verifica que:

1. La variable `v` sea declarada como array.
2. El índice `i` sea de tipo entero.
3. El `valor` asignado sea compatible con el tipo base (int/char).

2.2.8 Semántica de Funciones

Verificación de Declaración:
Toda variable, array o función debe haber sido declarada en la Tabla de Símbolos antes de su uso. Si no existe, se reporta error semántico.

Unicidad:
No se permiten identificadores duplicados (variables o funciones) dentro del mismo ámbito.

Llamadas:
Se verifica que el identificador invocado corresponda estrictamente a una función o procedimiento declarado previamente.

3 Estructuras Globales

3.1 Tabla de Símbolos

La Tabla de Símbolos es la estructura encargada de almacenar la información de las declaraciones. Su implementación permite la gestión de ámbitos (scopes) anidados mediante una clase interna `Ambito`.

Para la gestión de la Tabla de Símbolos he decidido usar un estilo de **Pila de Tablas Hash** (*Stack of HashMaps*). Para ello me he respaldado de documentación externa, (Ver documento sobre Tablas de Símbolos) este esquema organiza los ámbitos mediante una estructura LIFO (Last-In, First-Out):

1. **Estructura:** La tabla global es una pila (`Stack`) donde cada nivel representa un ámbito activo.
2. **Almacenamiento:** Cada elemento de la pila es una tabla de dispersión independiente (`HashMap`) que contiene únicamente los símbolos declarados en ese bloque.
3. **Operativa:**
   - Al entrar en un bloque (`entraBloc`), se apila una nueva tabla hash vacía en la cima.
   - La inserción de símbolos se realiza siempre en la tabla de la cima (ámbito local).

- La búsqueda se realiza iterativamente desde la cima hacia la base de la pila, garantizando la regla de alcance estático y la ocultación de variables (*shadowing*).
- Al salir del bloque (`salirBloc`), se desapila la tabla de la cima, liberando automáticamente toda la memoria asociada a las variables locales de ese ámbito.

Esta implementación aprovecha las capacidades de los lenguajes orientados a objetos modernos.

3.1.1 Clase de Información: `Descripcion`

En lugar de un nodo genérico, la tabla almacena objetos de la clase `Descripcion`. Cada descripción contiene la información semántica y de generación de código:

Atributo	Tipo Java	Descripción
`categoria`	`Categoria`	Rol del identificador (`DVAR`, `DPROC`, etc.).
`tsb`	`TSB`	Tipo de dato (`TS_ENTER`, `TS_BOOL`...).
`nivel`	`int`	Nivel de profundidad de los datos
`direccion`	`int`	Dirección relativa (offset) para variables locales.
`tamano`	`Integer`	Tamaño en elementos (solo para Arrays).
`ocupVL`	`int`	Ocupación total de variables locales (solo para Funciones).
`listaTiposArgs`	`List<TSB>`	Firma de los argumentos para validar llamadas.

3.1.2 Organización: Encadenamiento de Ámbitos

La clase `TablaSimbolos` gestiona una lista enlazada de ámbitos. Se utiliza una clase interna privada `Ambito` que actúa como nodo de la lista:

- **tabla:** `HashMap<String, Descripcion>`. Diccionario de símbolos del bloque actual.
- **anterior:** `Ambito`. Puntero al ámbito padre (scope superior).

3.1.3 Operaciones Principales

- **entraBloc():** Crea un nuevo objeto `Ambito`, lo enlaza con el actual y mueve el puntero `ambitoActual` hacia abajo (profundidad ++).
- **salirBloc():** Restaura el puntero `ambitoActual` al nodo `anterior` (profundidad –), liberando la memoria del bloque que se cierra.
- **poner(id, desc):** Inserta un símbolo en el `HashMap` del ámbito actual. Retorna `false` si la clave ya existe en este nivel (re-declaración ilegal).

3.1.4 Algoritmo de Búsqueda (`get`)

Para resolver un identificador, se implementa una búsqueda iterativa ascendente:

1. Se consulta el `HashMap` del ámbito actual.
2. Si se encuentra, se retorna la `Descripcion`.

3. Si no, se actualiza el puntero al ámbito `anterior` (Padre) y se repite el paso 1.
4. Si el puntero es `null` (se ha superado el ámbito global), se retorna `null`.

Se realiza de forma **ascendente**, ya que, debemos encontrar primero la declaración más cercana al punto de ejecución. Si buscáramos de forma descendente, encontraríamos siempre las variables globales antes que las locales, impidiendo que una función pueda tener sus propias variables privadas con nombres comunes.

4 Front-End (Análisis)

4.1 Análisis Léxico (Scanner) con JFlex

Para la implementación del analizador léxico se ha utilizado la herramienta **JFlex**, un generador de escáneres léxicos para Java. Esta decisión permite definir los tokens mediante expresiones regulares, delegando la gestión del buffer de entrada y la complejidad del autómata a la herramienta generada.

4.1.1 Especificación Léxica (.flex)

El comportamiento del escáner se define en un fichero de especificación ('scanner.flex') que consta de tres partes:

1. **Código de Usuario:** Definiciones de paquetes e importaciones (ej. la clase 'Token').
2. **Opciones y Declaraciones:** Se configura JFlex para que soporte la cuenta de líneas y columnas (`%line`, `%column`) y se definen macros para las expresiones regulares recurrentes (como `Digito` o `Lletra`).
3. **Reglas Léxicas:** Asocian un patrón (Expresión Regular) con una acción (Código Java).

4.1.2 Definición de Patrones

Se han definido macros para simplificar las reglas, siguiendo el diseño del lenguaje, por ejemplo:

```
Lletra   = [A-Za-z_]
Digit    = [0-9]
Alphanum = {Lletra}|{Digit}
Entero   = 0 | [1-9]{Digit}*
Id       = {Lletra}({Alphanum})*
```

Listing 1: Macros en JFlex

4.1.3 Gestión de Tokens y Valores

Cuando JFlex reconoce un patrón, ejecuta la acción asociada. En nuestra implementación, cada acción devuelve un objeto `Symbol` compatible con CUP. Por ejemplo:

```
    "if"        { return new Symbol(sym.IF, yyline, yycolumn,
        yytext()); }
```

Listing 2: Reglas y Acciones JFlex

4.1.4 Gestión de Errores Léxicos

JFlex incluye una regla por defecto ('.') que captura cualquier carácter que no coincida con las reglas anteriores. Se utiliza este mecanismo para invocar al **Gestor de Errores**:

```
. {
    GestorErrores.errorLexico((yyline+1), (yycolumn+1), "Caracter
        ilegal: " + yytext());
}
```

4.2 Análisis Sintáctico (Parser)

Para la fase de análisis sintáctico se ha optado por utilizar la herramienta **CUP** (Constructor of Useful Parsers), un generador de analizadores sintácticos LALR(1) para Java. Esto permite una:

- **Automatización**: Permite definir la gramática en un formato de alto nivel (.cup).
- **Gestión de Errores**: CUP incorpora mecanismos robustos de recuperación.
- **Integración**: Se integra de forma nativa con JFlex.

La elección de un generador basado en el algoritmo LALR(1) (*Look-Ahead LR*) frente a otras alternativas como LL(1) (descendentes) o LR(1) canónico (ascendentes puros) debe a lo siguiente:

1. **Soporte de Recursividad por la Izquierda:** A diferencia de los analizadores LL(top-down), LALR soporta producciones recursivas por la izquierda (ej. `E ::= E + T`). Esto permite definir las expresiones aritméticas de forma natural y directa, facilitando la implementación de la precedencia y asociatividad de operadores sin necesidad de transformar la gramática a formas complejas (eliminación de recursividad izquierda).
2. **Eficiencia de la Tabla de Análisis:** Aunque el algoritmo LR(1) es más potente, genera tablas de estados muy grandes para lenguajes reales. LALR(1) fusiona los estados que tienen el mismo núcleo (core) pero diferente símbolo de anticipación (lookahead), reduciendo drásticamente el tamaño de la tabla de parsing sin sacrificar prácticamente capacidad de reconocimiento para lenguajes de programación estructurados.
3. **Resolución de Conflictos:** El algoritmo permite resolver ambigüedades gramaticales comunes, como el problema del 'if-else' (si un `else` pertenece al `if` más cercano o al externo), mediante preferencias de desplazamiento sobre reducción, simplificando el diseño del lenguaje.

4.2.1 Mejoras en la Interfaz de Errores CUP

Por defecto, la herramienta CUP genera mensajes de error genéricos en inglés o vuelca la pila de excepciones. Para mostrarlos en español, se ha sobrescrito el método interno `syntax_error(Symbol s)` del parser con las siguientes funcionalidades:

1. **Intercepción del error:** Se captura el evento de error antes de que CUP imprima cualquier mensaje en la consola.
2. **Cálculo de tokens esperados:** Mediante el método `expected_token_ids()`, el compilador determina qué tokens eran válidos en el punto de fallo.
3. **Traducción automática:** Se mapean los nombres internos de los tokens (por ejemplo, `LPAREN`, `ID`) a nombres legibles para el programador (como `'('` o `un identificador`),

generando mensajes de ayuda contextuales, por ejemplo: *Se esperaba: un número, un identificador*.

4. **Ubicación precisa:** Para errores en producciones específicas, se utilizan las coordenadas del último token válido (`idleft`, `idright`) para evitar el problema de coordenadas `(0,0)`.

4.2.2 Estrategias de Recuperación de Errores (Panic Mode)

Además de mejorar los mensajes, se ha implementado una estrategia de recuperación de errores. Esto permite al compilador descartar instrucciones defectuosas y resincronizarse con tokens delimitadores conocidos, evitando detener el análisis ante el primer fallo. Se han definido puntos de recuperación específicos en:

- **Declaraciones Globales:** Si una declaración falla, se descarta hasta el siguiente punto y coma.
- **Cabeceras de Bucles:** En estructuras `for`, si la inicialización o condición son erróneas, se asume una estructura válida por defecto para intentar analizar el cuerpo del bucle.
- **Sentencias Switch:** Errores en un `case` no invalidan el resto de casos.

4.2.3 Construcción del AST y Declaraciones Múltiples

Una complejidad abordada en la gramática es la gestión de declaraciones múltiples en una sola línea (ej. `int a, b = 5, c[10];`). Dado que el AST requiere un nodo individual por cada variable declarada para simplificar el análisis semántico y la generación de código:

1. La gramática utiliza variables auxiliares en el `action code` (declaracionesLinea) para acumular las variables encontradas en una sentencia.
2. Al finalizar la instrucción (token `PUNTO_COMA`), el parser devuelve un `NodoBloque` sintético que contiene la lista secuencial de los nodos `NodoDeclaracion` individuales.

4.2.4 Precedencia de Operadores

Para mantener la gramática compacta y legible, se ha optado por definir la precedencia y asociatividad de los operadores de forma declarativa (usando las directivas `precedence left/right` de CUP) en lugar de tener una gramática con múltiples niveles de no-terminales (como *Termino*, *Factor*, etc.). Reduciendo la profundidad del árbol de derivación y mejorando ligeramente el rendimiento del parser.

4.3 Representación Intermedia: Árbol Sintáctico Abstracto (AST)

El resultado de la fase de análisis sintáctico no es simplemente la validación de la gramática, sino la construcción de una estructura de datos jerárquica que representa el programa en memoria: el **Árbol Sintáctico Abstracto (AST)**. A diferencia del árbol de derivación concreto (que contiene todos los tokens y reglas gramaticales), el AST simplifica la estructura eliminando detalles sintácticos innecesarios (como paréntesis, puntos y coma o palabras clave) y centrándose en la estructura lógica de las operaciones.

4.3.1 Diseño de Clases del AST

Se ha implementado una jerarquía de clases en Java bajo el paquete `ArbolSintactico`, utilizando el polimorfismo para recorrer y procesar el árbol en fases posteriores.

- **NodoBase:** Clase abstracta padre de todos los nodos. Almacena la información de depuración (`línea` y `columna`) para el reporte de errores semánticos.
- **NodoSentencia:** Representa instrucciones que no retornan valor (control de flujo, declaraciones).
- **NodoExpresion:** Representa construcciones que evalúan a un tipo de dato (operaciones, literales, llamadas).

4.3.2 Nodos Implementados

La jerarquía de clases del AST se compone de nodos abstractos (estructurales) y nodos concretos. A continuación se detallan todos los nodos implementados:

A. Clases Abstractas y Estructurales

Clase	Descripción
`NodoBase`	Clase raíz de toda la jerarquía. Almacena la ubicación del código fuente (línea y columna) para el reporte de errores.
`NodoSentencia`	Clase base para instrucciones que ejecutan acciones y modifican el estado (control de flujo, asignaciones), pero no tienen valor asociado.
`NodoExpresion`	Clase base para construcciones que se evalúan a un valor y tienen un tipo de dato asociado (Tipado).
`NodoSentenciaLlamada`	Nodo adaptador que encapsula un `NodoLlamada`. Permite que una invocación a función (que es una expresión) actúe sintácticamente como una sentencia independiente (ej. `funcion();`).

B. Sentencias y Control de Flujo

Nodo	Descripción
`NodoBloque`	Contenedor de una lista secuencial de sentencias (ámbito).
`NodoDeclaracion`	Define una variable y registra su tipo en la Tabla de Símbolos.
`NodoAsignacion`	Asigna el resultado de una expresión a una variable simple.
`NodoAsignacionArray`	Escritura en una posición de un vector (`vec[i] = val`).
`NodoIf / NodoWhile`	Estructuras de control de flujo condicionales e iterativas.
`NodoFor`	Bucle iterativo completo con inicialización, condición y paso.
`NodoSwitch / NodoCaso`	Estructura de selección múltiple y definición de ramas.
`NodoFuncion`	Definición de un subprograma. Se utiliza tanto para funciones con retorno como para procedimientos (`void`), distinguiéndose por su atributo de tipo.
`NodoReturn`	Sentencia de retorno (finalización) de función.
`NodoLeer / NodoImprimir`	Sentencias de Entrada (Input) y Salida (Output).

C. Expresiones y Valores

Nodo	Descripción
`NodoOperacion`	Operaciones binarias (Aritméticas, Lógicas y Relacionales).
`NodoUnario`	Operaciones con un solo operando (Negación `!` y Menos `-`).
`NodoLlamada`	Invocación a una función que retorna un valor.
`NodoAccesoArray`	Acceso de lectura indexado a vectores (`vec[i]`).
`NodoLiteral`	Hoja que representa un valor constante (número, char, bool).
`NodoIdentificador`	Hoja que representa el uso de una variable existente.

Por ejemplo, al usar un *if* se generaria el siguiente arbol:

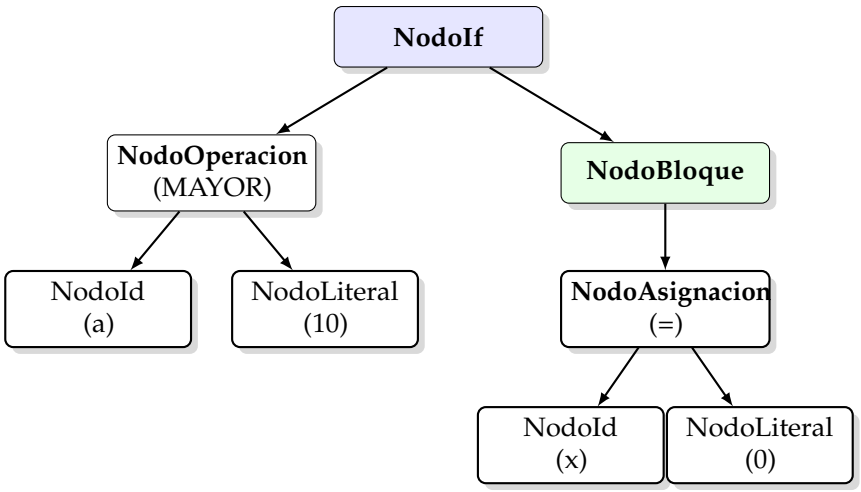

Figura 2: Representación del AST para la sentencia: `if (a >10) { x = 0; }`

4.3.3 Estrategia de Construcción: Flattening (Aplanamiento)

Para evitar la creación de árboles excesivamente profundos y anidados (bloques dentro de bloques innecesarios producidos por la recursividad de la gramática), se ha implementado una estrategia de 'aplanamiento' en el Parser.
Al insertar una lista de declaraciones o sentencias en un bloque padre, si el elemento a insertar es a su vez un `NodoBloque`, se extraen sus sentencias internas y se añaden directamente a la lista del padre. Esto genera una estructura lineal mucho más fácil de recorrer por el Analizador Semántico y el Generador de Código.

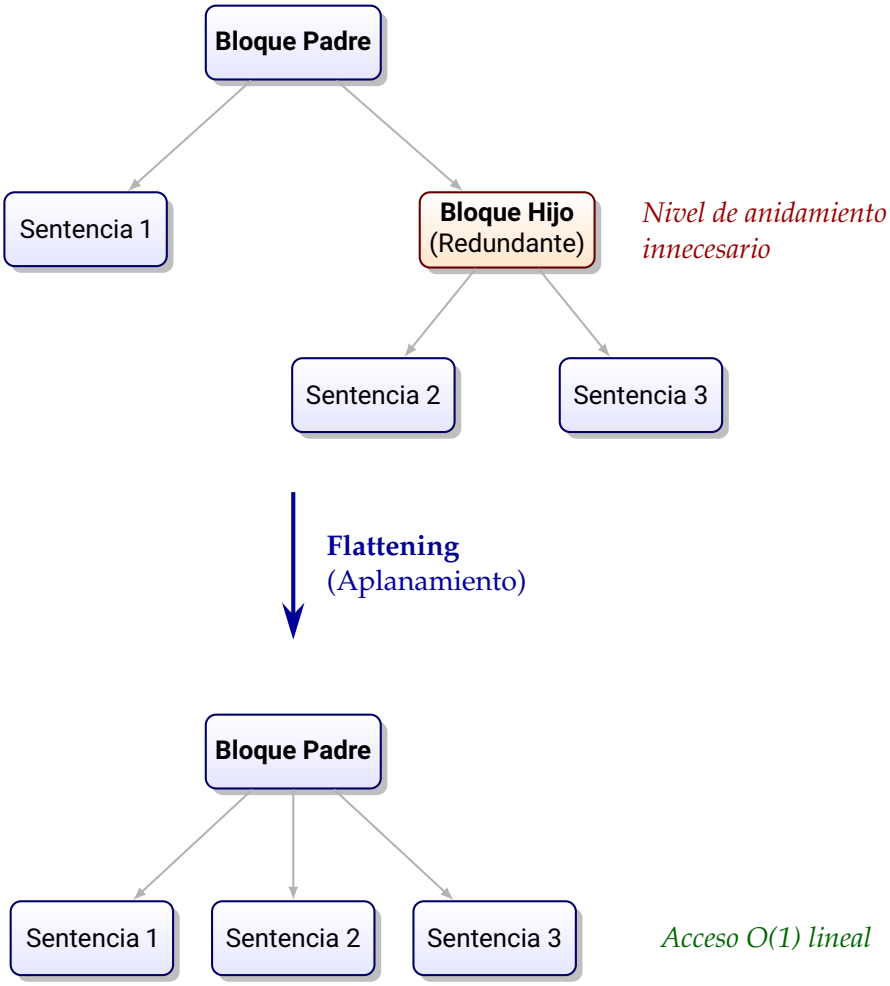

Figura 3: Optimización estructural del AST mediante Flattening.

4.4 Análisis Semántico

Esta fase se encarga de verificar la coherencia del programa utilizando la información estructural obtenida del análisis sintáctico. El análisis se basa en una **Gramática de Atributos**, donde cada nodo del árbol sintáctico se decora con información semántica (tipos, modos de acceso, etc.).

4.4.1 Sistema de Tipos: Tipos Subyacentes Básicos (TSB)

Para gestionar la compatibilidad entre tipos, se ha implementado el concepto de *Tipo Subyacente Básico* (TSB).

4.4.2 Estructura de la Tabla de Símbolos

La Tabla de Símbolos es la estructura central que almacena las declaraciones. Cada entrada en la tabla contiene una Descripción ('td') que categoriza el identificador.

Se ha definido una jerarquía de clases para representar los diferentes tipos de identificadores en el *package* de Símbolos.

4.4.3 Gestión de Ámbitos (Scopes)

Para soportar la visibilidad y ocultación de variables, la tabla de símbolos implementa las operaciones 'entrabloc' y 'sortirbloc' ya descritas. Se utiliza una estructura basada en una tabla de expansión ('te') y una tabla de descripciones ('td') para gestionar la visibilidad actual de cada identificador.

4.5 Implementación del Analizador Semántico

La validación semántica se realiza en la clase `AnalizadorSemantico`, la cual recorre el AST en profundidad. Además de la verificación de tipos, esta clase tiene una responsabilidad crítica: el cálculo de direcciones de memoria (offsets) para las variables, información indispensable para la posterior generación de código ensamblador.

4.5.1 Cálculo de Direcciones de Memoria (Offset)

El analizador mantiene un contador interno, `desplazamientoActual`, que simula el puntero de pila durante la compilación:

- **Variables Globales y Locales:** Cada vez que se procesa una declaración (`procesarDeclaracion`), se asigna el desplazamiento actual a la variable y se incrementa el contador según su tamaño (4 bytes para enteros/booleanos/chars).
- **Argumentos de Función:** Al entrar en una función, se reinicia el desplazamiento para las variables locales, pero se asignan offsets fijos positivos (empezando en 8) a los parámetros. Esto respeta la llamada de la arquitectura x86 (donde los argumentos están 'arriba' del *Base Pointer*).

Esta información se exporta al Backend mediante dos estructuras estáticas:

- `globalesDeclaradas`: Mapea nombres de variables globales a su tamaño.
- `argumentosOffset`: Mapea el par *Función_Parámetro* a su posición en la pila.

4.5.2 Compatibilidad y Coerción de Tipos

Se ha implementado cierta flexibilidad para tipos numéricos. El método auxiliar `sonCompatibles(t1, t2)` permite la corrección implícita entre `INT` y `CHAR`, permitiendo operaciones aritméticas con caracteres sin necesidad de cast explícito, comportándose como en C.

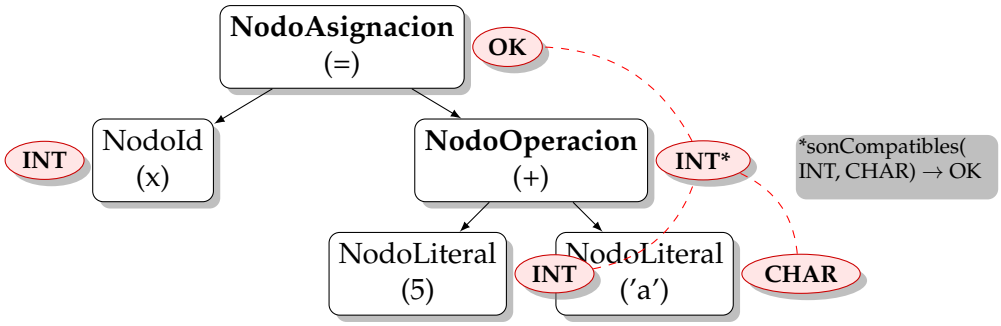

Figura 4: Flujo de atributos sintetizados durante la evaluación de expresiones. Se muestra la coerción implícita de `char` a `int`.

4.5.3 Estrategia de Recuperación de Errores en Expresiones

Para evitar el efecto de 'cascada de errores' (donde un fallo semántico provoca cientos de errores falsos a continuación), el método `evaluarExpresion` implementa una recuperación inteligente: Si se detecta una incompatibilidad en una operación (ej. `true + 5`), el gestor reporta el error, pero el método retorna el tipo esperado del resultado (en este caso `TS_ENTER`) en lugar de un tipo de error. Esto permite que los nodos padre sigan analizando el resto del árbol asumiendo que la operación hija fue correcta.

4.5.4 Validaciones Específicas Implementadas

- **Arrays:** Se valida que el índice sea estrictamente entero y se impide la asignación directa de arrays completos (solo acceso elemento a elemento).
- **Switch:** Se restringe el discriminante a tipos discretos (`INT` o `CHAR`) y se verifica que todos los casos sean compatibles con él.
- **Funciones:** Se comprueba el número de argumentos y la concordancia de tipos en cada llamada. Además, se detectan parámetros duplicados en la definición.

4.5.5 Análisis de Sentencias y Evaluación de Expresiones

- **analizarSentencia(NodoSentencia s):** Procesa nodos que no retornan valor (declaraciones, `if`, `while`). Su objetivo es actualizar la Tabla de Símbolos o controlar el flujo del análisis (entrar/salir de bloques).
- **evaluarExpresion(NodoExpresion e):** Procesa nodos que representan un valor. Su objetivo es retornar el `TSB` (Tipo Subyacente Básico) resultante de la expresión para verificar compatibilidades. Por ejemplo, ante una suma, evalúa los operandos izquierdo y derecho; si ambos son `INT`, retorna `INT`; si son incompatibles, reporta error y retorna `TS_ERROR`.

4.5.6 Procesamiento de Funciones y Parámetros

Uno de los puntos críticos implementados es la validación de subprogramas. El proceso sigue un orden estricto para garantizar que los parámetros y variables locales no entren en conflicto y sean visibles correctamente:

1. **Registro de la Firma:** Se inserta el nombre de la función y su tipo de retorno en el ámbito actual (Global) para permitir su invocación.
2. **Apertura de Ámbito:** Se invoca `ts.entraBloc()` para crear un nuevo entorno local.
3. **Registro de Parámetros:** Se recorre la lista de argumentos del `NodoFuncion`. Cada parámetro se trata semánticamente como una declaración de variable local en el nuevo ámbito. Esto permite detectar errores como parámetros duplicados (`void f(int a, int a)`).
4. **Análisis del Cuerpo:** Se analizan las sentencias internas de la función.
5. **Cierre de Ámbito:** Se invoca `ts.salirBloc()` para destruir el contexto local y volver al global.

4.5.7 Detección de Errores

El analizador utiliza el `GestorErrores` para reportar incidencias semánticas sin detener abruptamente la compilación (salvo errores fatales), permitiendo detectar múltiples fallos en una sola pasada (ej. variables no declaradas, tipos incompatibles en asignaciones y funciones no definidas).

5 Back-End (Síntesis)

5.1 Generación de Código Intermedio

El proceso de compilación pasa de la fase de análisis (Front-End) a la fase de síntesis (Back-End). La generación de código intermedio define la interfaz entre estas dos fases, permitiendo que un mismo Front-End sea compatible con múltiples arquitecturas de destino y facilitando la optimización independiente de la máquina.

5.1.1 El Modelo: Código de Tres Direcciones (C3@)

Se ha seleccionado el modelo de **Código de Tres Direcciones (C3@)** por ser la representación que simplifica las operaciones complejas del lenguaje fuente en secuencias de operaciones elementales.

Características del Modelo

- **Abstracción:** Es independiente del lenguaje ensamblador final y no utiliza registros físicos de la CPU, sino variables temporales y nombres simbólicos.
- **Estructura:** Cada instrucción consta, como máximo, de un operador y tres direcciones (dos operandos y un destino).
- **Identificación:** Todos los elementos del programa (variables, temporales, subprogramas) se identifican mediante códigos numéricos únicos (IDs enteros), eliminando la dependencia de los nombres de cadena (strings).

La estructura lógica de cada instrucción es:

```
CÓDIGO_OPERACIÓN   OPERANDO_1   OPERANDO_2   DESTINO
```

5.1.2 Estructuras de Datos Auxiliares

Para gestionar la generación de código, es necesario mantener dos tablas que complementan a la Tabla de Símbolos del Front-End. Estas tablas almacenan la información necesaria para la ubicación en memoria y la gestión de llamadas.

5.1.3 Tabla de Variables (TV)

Esta estructura almacena los descriptores de todas las variables declaradas. En esta implementación, la TV no es una estructura separada, sino que está integrada en la **Tabla de Símbolos** general.

Cuando un símbolo se identifica con la categoría `DVAR` (o `DARG`), se utilizan los siguientes atributos de la clase `Descripcion` para definirlo:

Atributo Java	Tipo	Significado Semántico
`nivel`	Entero	**Ámbito de declaración**: Identifica si la variable es global (0), pertenece al `main` (1) o a una función específica.
`tsb`	Enum	**Tipo de Dato**: Indica el Tipo Subyacente Básico (`TS_ENTER`, `TS_CHAR`, etc.).
`direccion`	Entero	**Offset / Desplazamiento**: Dirección de memoria relativa respecto al puntero de base (`%ebp`).
`tamano`	Entero	**Dimensión (Arrays)**: Si es un array, almacena el número de elementos declarados.

Nota de Diseño: Alineación de Memoria Por simplicidad en la generación de código para la arquitectura IA32 y para mantener la alineación de la pila (*Stack Alignment*), se ha decidido que **todos los tipos primitivos (`int`, `char`, `bool`) ocupen 4 bytes (32 bits)**. Esto simplifica el cálculo de desplazamientos (siempre múltiplos de 4) y el uso de las instrucciones de ensamblador `pushl/popl`, aunque implique un ligero desperdicio de memoria en tipos pequeños.

5.1.4 Tabla de Procedimientos (TP)

Al igual que la TV, esta estructura no es independiente, sino que se integra en la **Tabla de Símbolos**. Se utiliza para almacenar la información de las funciones y procedimientos declarados (`Categoria.DPROC`).

Los atributos de la clase `Descripcion` relevantes para esta tabla son:

Atributo Java	Tipo	Significado Semántico
`nivel`	Entero	**Profundidad**: Nivel de anidamiento (0 para global).
`tsb`	Enum	**Tipo de Retorno**: El tipo de dato que devuelve la función (`TS_ENTER`, `TS_VOID`, etc.).
`ocupVL`	Entero	**Ocupación Local**: Suma total (en bytes) que ocupan todas las variables declaradas dentro de la función. Necesario para generar la instrucción `subl` en el preámbulo.
`listaTiposArgs`	Lista	**Firma**: Lista ordenada con los tipos (`TSB`) de los parámetros formales. Se usa para validar que las llamadas coincidan en número y tipo de argumentos.

5.1.5 Juego de Instrucciones C3@

A continuación se define el conjunto de instrucciones soportadas por el generador, clasificadas por su funcionalidad.

5.1.6 Operaciones Aritméticas y Lógicas

Operaciones binarias estándar y unarias. El resultado se almacena siempre en una variable destino.

Mnemónico	Formato	Descripción
`ADD`	`ADD op1 op2 dest`	Suma: $dest = op1 + op2$
`SUB`	`SUB op1 op2 dest`	Resta: $dest = op1 - op2$
`MUL`	`MUL op1 op2 dest`	Producto: $dest = op1 * op2$
`DIV`	`DIV op1 op2 dest`	División entera: $dest = op1/op2$
`MOD`	`MOD op1 op2 dest`	Módulo: $dest = op1\,\%op2$
`NEG`	`NEG op1 - dest`	Menos unario: $dest = -op1$
`AND/OR`	`OP op1 op2 dest`	Operaciones lógicas booleanas.
`NOT`	`NOT op1 - dest`	Negación lógica.
`COPY`	`COPY op1 - dest`	Asignación simple: $dest = op1$.

5.1.7 Entrada / Salida

Mnemónico	Formato	Descripción
`READ`	`READ - - dest`	Lee un entero por teclado y lo guarda en dest.
`READ_CHAR`	`READ_CHAR - - dest`	Lee un carácter por teclado y guarda su código ASCII en dest.
`PRINT`	`PRINT val - -`	Imprime el valor numérico de val (Enteros).
`PRINT_CHAR`	`PRINT_CHAR val - -`	Imprime el valor interpretado como carácter ASCII.

5.1.8 Gestión de Memoria (Arrays y Punteros)

Instrucciones necesarias para manejar accesos indexados a vectores ($v[i]$) y asignaciones indirectas.

Mnemónico	Formato	Descripción
`IND_VAL`	`IND_VAL b i d`	Lectura indexada (Indirection Value): $d = b[i]$.
`IND_ASS`	`IND_ASS v i b`	Escritura indexada (Indirection Assign): $b[i] = v$.

5.1.9 Control de Flujo (Saltos)

Instrucciones para implementar estructuras de control (`if`, `while`). Se utilizan etiquetas simbólicas (`skip`) como destinos de salto.

Mnemónico	Formato	Descripción
`SKIP`	`etiqueta: SKIP`	Define una etiqueta o punto de destino (LABEL).
`GOTO`	`GOTO dest`	Salto incondicional a la etiqueta `dest`.
`IF_EQ`	`IF_EQ o1 o2 dest`	Salta si $o1 == o2$.
`IF_NE`	`IF_NE o1 o2 dest`	Salta si $o1 \neq o2$.
`IF_LT`	`IF_LT o1 o2 dest`	Salta si $o1 < o2$.
`IF_GT`	`IF_GT o1 o2 dest`	Salta si $o1 > o2$.
`IF_LE`	`IF_LE o1 o2 dest`	Salta si $o1 \leq o2$.
`IF_GE`	`IF_GE o1 o2 dest`	Salta si $o1 \geq o2$.

5.1.10 Gestión de Subprogramas

Instrucciones específicas para la definición y llamada de funciones, siguiendo la convención de llamadas estándar (paso de parámetros y gestión de pila).

Mnemónico	Formato	Descripción
`PMB`	`PMB id`	**Preámbulo**: Marca el inicio de una función `id`. Indica al generador que debe crear el prólogo de la función (guardar `ebp`, reservar espacio para locales).
`PARAM_S`	`PARAM_S val`	Empila el parámetro `val` en la pila antes de realizar una llamada (`push`).
`CALL`	`CALL id, n, dest`	Llama a la función `id` con `n` parámetros. El resultado de la función (si no es void) se almacena en la variable temporal `dest`.
`RET`	`RET val?`	Retorno de subrutina. Opcionalmente devuelve el valor `val` (lo coloca en `%eax`) y restaura la pila.

5.2 Implementación del Generador de Código Intermedio

Para automatizar la creación del Código de Tres Direcciones, se ha implementado un nuevo recorrido del Árbol Sintáctico Abstracto (AST) mediante la clase `GeneradorCodigo`. Este módulo utiliza un patrón de diseño similar al del Analizador Semántico (Visitor), pero con un objetivo diferente: en lugar de validar tipos, su función es emitir instrucciones lineales equivalentes a la lógica del árbol.

5.2.1 Estrategia de Recorrido: Doble Pasada

Para garantizar la correcta generación de código ensamblador y evitar que el flujo de ejecución del programa principal (`main`) invada el código de las subrutinas, se ha implementado una estrategia de generación en **Dos Pasadas** sobre el nodo raíz del AST:

1. **Fase 1 - Generación de Subrutinas:** Se recorre la lista de sentencias globales buscando exclusivamente nodos de tipo `NodoFuncion`. Se genera su etiqueta, su preámbulo (`PMB`) y su cuerpo. Esto asegura que todo el código de funciones quede aislado al principio del segmento de texto.
2. **Fase 2 - Generación del Main:** Se recorre nuevamente el AST ignorando las funciones y procesando únicamente las declaraciones globales y el bloque principal. Se emite explícitamente la etiqueta `main:` y su preámbulo antes de generar las instrucciones, y se asegura el cierre con una instrucción de salida del sistema.

Aunque, por definición, el main siempre lo debemos de poner al final den nuestros programa, se optó por realizar una doble pasada, ya que durante la fase de pruebas con subprogramas, en ocasiones, el flujo de control del main invadía la declaración de los subprogramas. Al forzar esta separación en la generación, aseguramos que el `main` y las funciones residan en bloques de memoria diferentes, evitando la corrupción del marco de pila (*Stack Frame*) y previniendo errores críticos de segmentación (Segmentation Faults) en el ejecutable final."

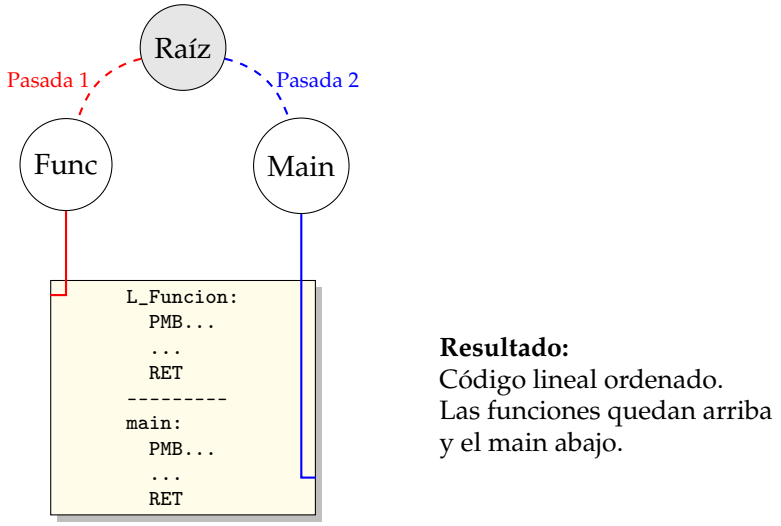

Figura 5: Estrategia de generación en dos pasadas para ordenar el segmento de código.

5.2.2 Arquitectura del Generador

El paquete `CodigoIntermedio` contiene las siguientes clases fundamentales:

- **TipoInstruccion (Enum):** Enumera todos los códigos de operación soportados (`ADD`, `SUB`, `GOTO`, `IF_EQ`, `CALL`, etc.).
- **Instruccion:** Representa una cuádrupla individual (Operación, Operando 1, Operando 2, Destino).
- **Generador:** Clase auxiliar encargada de la gestión de recursos. Mantiene los contadores para generar nombres únicos de:
  - **Variables Temporales:** $t0, t1, t2 \ldots$ (para resultados parciales de expresiones).
  - **Etiquetas:** $L0, L1, L2 \ldots$ (para destinos de saltos en estructuras de control).
- **GeneradorCodigo:** La clase principal que contiene la lógica de traducción recursiva.

5.2.3 Estrategia de Traducción

La traducción se realiza diferenciando entre expresiones (que retornan valor) y sentencias (que controlan flujo).

Normalización de Literales y Escapes. Antes de procesar operaciones complejas, el generador traduce los valores constantes. En el caso de los caracteres, se implementa una lógica de des-escape (*unescaping*) en la fase de generación: las secuencias como `'\n'`, `'\t'` o `'\0'` se transforman a su valor numérico ASCII correspondiente (10, 9, 0) antes de emitir la instrucción `COPY`. Esto permite que el ensamblador trabaje exclusivamente con valores enteros, simplificando la aritmética posterior.

1. Traducción de Expresiones Aritméticas
El método `tratarExpresion` recorre el árbol en post-orden. Para una operación binaria (ej. $a + b$):

1. Se generan las instrucciones para evaluar el operando izquierdo (guardado en $op1$).
2. Se generan las instrucciones para evaluar el operando derecho (guardado en $op2$).
3. Se solicita una nueva temporal (t_n).
4. Se emite la instrucción `ADD op1, op2, tn`.
5. Se retorna el nombre t_n al nodo padre.

2. Traducción de Expresiones Relacionales (Síntesis)
Dado que el C3@ no maneja valores booleanos complejos directamente en comparaciones, expresiones como `x = a <b` se sintetizan utilizando saltos:

```
    IF_LT a, b, L_True
    x = 0
    GOTO L_Fin
L_True:
    x = 1
L_Fin:
    ...
```

3. Traducción de Estructuras de Control (If / While)
Para las sentencias de control, el generador crea etiquetas de antemano y utiliza instrucciones de salto condicional e incondicional. *Ejemplo de traducción para un bucle While:*

1. Se genera y emite una etiqueta de `Inicio`.

2. Se evalúa la condición.
3. Se genera una instrucción `IF_EQ condicion, 0, Fin` (si es falsa, salta al final).
4. Se genera el código del cuerpo del bucle.
5. Se emite un `GOTO Inicio`.
6. Se emite la etiqueta `Fin`.

Traducción del Bucle For: A diferencia del `while`, el bucle `for` requiere gestionar cuatro componentes: inicialización, condición, cuerpo e incremento.

1. Se emite el código de **inicialización**.
2. Se genera la etiqueta `Inicio`.
3. Se evalúa la **condición** y se salta a `Fin` si es falsa.
4. Se emite el código del **cuerpo** del bucle (recursivamente).
5. Se emite el código de **incremento**.
6. Se emite un `GOTO Inicio` y finalmente la etiqueta `Fin`.

Traducción de la Sentencia Switch: El `switch` se traduce convirtiéndolo en una secuencia de comparaciones y saltos explícitos.

1. Se evalúa la expresión discriminante.
2. Se genera una serie de instrucciones `IF_EQ` comparando el discriminante con el valor de cada `case`. Si coincide, se salta a la etiqueta específica de ese bloque.
3. Si ninguna comparación tiene éxito, se salta a la etiqueta de salida (o `default`).
4. A continuación se emiten las etiquetas y los bloques de código de cada caso, asegurando que el flujo converja en la etiqueta final.

   Se ha optado por traducir la estructura `switch` mediante una secuencia lineal de saltos condicionales (**Cascada de IFs**) en lugar de utilizar una *Jump Table* (Tabla de Saltos) o búsqueda binaria. Aunque la complejidad temporal es lineal $O(N)$ frente al $O(1)$ de la tabla de saltos, esta estrategia simplifica drásticamente la generación de código intermedio, facilita la depuración y es suficientemente eficiente para el número de casos habitual en el que se va a someter el compilador.

4. Traducción de Funciones
Al procesar un nodo `NodoFuncion`, se emite una etiqueta con el nombre de la función y una instrucción especial `PMB` (Preámbulo), encargada de indicar al Back-End que debe generar el código de gestión de la pila (Stack Frame) y reserva de espacio para variables locales.

Traducción de Llamadas (Convención cdecl):
Para realizar una llamada a función, se evalúan los argumentos y se emiten instrucciones `PARAM_S`. Un aspecto crítico implementado es el **orden inverso de empilado**: los argumentos se procesan de derecha a izquierda (del último al primero). Esto asegura que, en tiempo de ejecución, el primer argumento se encuentre en la posición `8(%ebp)`, el segundo en `12(%ebp)`, etc., cumpliendo con la convención de llamada estándar de C en x86.

5. Traducción de Arrays
La gestión de vectores requiere distinguir entre el acceso de lectura y la escritura, generando instrucciones de direccionamiento indirecto:

- **Escritura (`vector[i] = valor`):** Se procesa como una sentencia. Se evalúa la expresión del índice (t_{ind}) y la del valor (t_{val}). Finalmente, se emite la instrucción `IND_ASS t_val, t_ind, nombre_array`.
- **Lectura (`x = vector[i]`):** Se procesa como una expresión. Se evalúa el índice (t_{ind}), se reserva una nueva variable temporal (t_{dest}) y se emite `IND_VAL nombre_array, t_ind, t_dest`. El temporal t_{dest} es el que se propaga hacia arriba en el árbol de expresión.

5.3 Generación de Código Ensamblador

La fase final del compilador traduce las instrucciones abstractas del Código de Tres Direcciones (C3@) al lenguaje máquina de la arquitectura destino.

5.3.1 Arquitectura de Destino

Se generará código para la arquitectura **IA32** (Intel 32-bits) utilizando la sintaxis **AT&T** (estándar de GNU/Linux), que se caracteriza por:

- Los registros llevan el prefijo % (ej. `%eax`).
- Los literales llevan el prefijo $ (ej. `$10`).
- El orden de los operandos es: `instrucción origen, destino`.

5.3.2 Mapeo de Memoria y Registros

En la versión actual, no se realiza una asignación optimizada de registros (Register Allocation). Se utiliza una estrategia de "traducción directa":

- Las variables del C3@ se mapean a direcciones de memoria en la Pila (Stack) relativas al `base pointer` (`%ebp`).
- Los registros (`%eax`, `%ebx`, etc.) se usan solo como almacenamiento temporal para realizar la operación aritmética e inmediatamente se devuelve el resultado a memoria.

5.3.3 Resolución de Direcciones (Linear Scan)

A diferencia de las fases anteriores, el generador de ensamblador no requiere reconstruir la estructura de árbol ni los ámbitos. Dado que el Código de Tres Direcciones ya ha linealizado el flujo de ejecución, el generador realiza un recorrido secuencial (*Linear Scan*). La gestión de variables se simplifica mediante un mapa único de desplazamientos (`offsetsTemporales`) que asigna espacio en la pila bajo demanda a cualquier identificador o temporal que aparezca por primera vez, garantizando que cada variable tenga una dirección física única dentro del marco de pila (*Stack Frame*) de la función actual.

5.4 Implementación del Generador de Ensamblador

La clase final del compilador, `GeneradorEnsamblador`, toma la lista de instrucciones C3@ y produce el archivo de texto `.s` compatible con el ensamblador.

5.4.1 Gestión de Memoria en el Backend

Uno de los mayores retos es la asignación de direcciones físicas a las variables temporales generadas en la fase anterior (t0, t1...), las cuales no existen en la Tabla de Símbolos original. Para resolver esto, se utiliza un mapa dinámico (`offsetsTemporales`) durante la generación:

1. **Variables Locales y Temporales:** Se mantiene un contador `currentOffsetLocal` (negativo) para asignar espacio en la pila a las variables nuevas.
2. **Argumentos de Función:** Se utiliza un mapa estático de comunicación entre el Analizador Semántico y el Generador de Ensamblador. Durante el análisis semántico de una función, se calculan y guardan los offsets positivos (`8, 12, ...`) de los parámetros. El backend consulta este mapa para resolver direcciones como `8(%ebp)`.
3. **Alineación de Pila:** En el preámbulo de funciones y del main, se asegura que el espacio reservado en la pila (`subl $N, %esp`) mantenga la alineación a 16 bytes requerida por la ABI de sistemas Linux modernos para evitar fallos en llamadas a librerías externas.

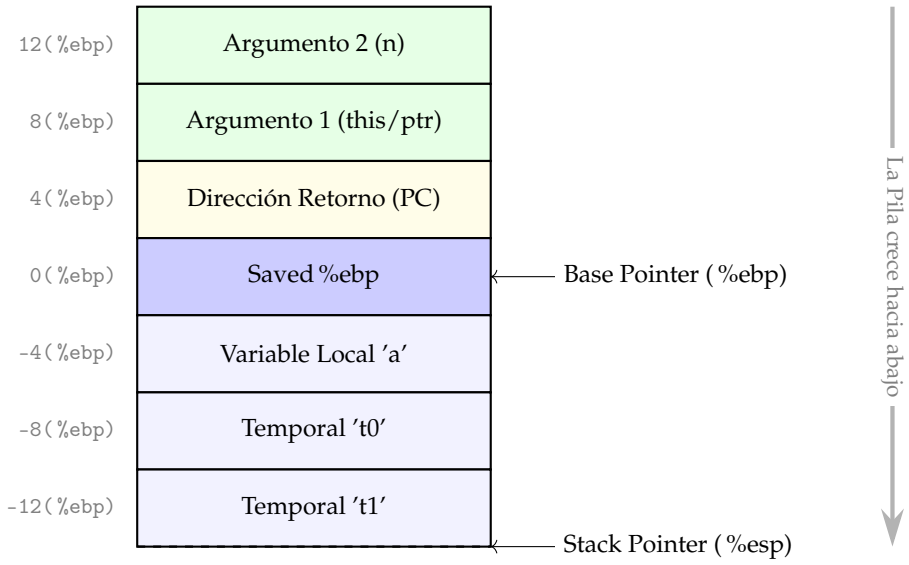

Figura 6: Diseño del Marco de Pila (Stack Frame) en arquitectura IA32.

5.4.2 Traducción de Instrucciones

La traducción se realiza mediante un mapeo directo de instrucciones C3@ a bloques de ensamblador IA32. Se han tenido en cuenta las particularidades de la sintaxis AT&T:

- **Orden de operandos:** `instrucción, origen, destino`.
- **Prefijos:** Uso de `$` para literales y `%` para registros.
- **Saltos:** Conversión de `IF_LT` (menor) a la secuencia `cmpl` + `jl` (Jump if Less).
- **Direccionamiento de Arrays:** Se utiliza el modo de direccionamiento indexado con escala de la arquitectura IA32: `base(índice, escala)`. Dado que el lenguaje trabaja con enteros de 32 bits (4 bytes), la traducción de `IND_VAL/ASS` genera instrucciones del tipo `movl (%edx, %edi, 4), %eax`, donde el hardware calcula automáticamente la dirección efectiva como $Base + (Indice \times 4)$.

También, podemos comentar el cálculo de direcciones base, el generador distingue dinámicamente la ubicación del array para cargar su dirección base en el registro de índice:

- Si el array es Global: Su nombre es una etiqueta de memoria, por lo que se carga la dirección con `movl $nombre,%reg`.
- Si el array es Local/Parámetro: Reside en la pila (ej. `-8(%ebp)`). Para obtener su dirección de memoria real (y no su valor), se utiliza la instrucción `leal` (Load Effective Address), por ejemplo: `leal -8(%ebp),%reg`.

5.4.3 Integración con C (Runtime)

Dado que el lenguaje ensamblador carece de funciones de alto nivel para entrada/salida, se ha optado por un enfoque mixto. El compilador genera llamadas a etiquetas externas que son resueltas en tiempo de enlace (*linking*) con una librería de tiempo de ejecución (`runtime.c`) escrita en C. Las funciones implementadas son:

- `print_int(int n)`: Imprime números enteros.
- `print_char(int c)`: Imprime el carácter correspondiente al código ASCII recibido.
- `read_int()`: Lee un entero desde la entrada estándar.
- `read_char()`: Lee un carácter de la entrada estándar, gestionando el buffer de teclado.

5.4.4 Decisión de Diseño: Runtime en C vs Llamadas al Sistema (Syscalls)

Tambíen se ha considerado implementar las operaciones de Entrada/Salida (E/S) utilizando directamente llamadas en lenguaje ensamblador.

Sin embargo, se optó por usar una librería externa(`runtime.c`) por las siguientes razones:

1. **Portabilidad Multi-Plataforma:** Las llamadas al sistema son dependientes del sistema operativo. El código ensamblador para imprimir en pantalla es diferente e incompatible entre Linux, Windows y macOS. Al utilizar la función `printf` de la librería estándar de C, se delega la gestión de la plataforma al compilador GCC durante la fase de enlace, permitiendo que nuestro compilador genere un código ensamblador que funciona indistintamente en Windows, Linux y MacOS.
2. **Complejidad de Formato:** La conversión de datos binarios requiere grandes rutinas de ensamblador (división iterativa por 10, gestión de signo, etc.) que aumentarían la dificultad del proyecto.

6 Guía de Usuario

El proyecto incluye scripts automatizados para facilitar la compilación y ejecución tanto en entornos Unix/Linux como en Windows.

6.1 Requisitos Previos

Para poder ejecutar correctamente el compilador, se requiere el siguiente software instalado:

- **Java JDK:** Necesario para compilar y ejecutar las herramientas del Front-End (JFlex y CUP) y el propio compilador.
- **GCC (GNU Compiler Collection):** Herramienta esencial para la fase de *back-end*. Dado que el compilador genera código en lenguaje ensamblador (`.s`), se requiere GCC para realizar los pasos finales de la construcción del programa:
  1. **Compilación del Runtime:** Compila la librería de soporte escrita en C (`runtime.c`) que contiene las funciones auxiliares del sistema.
  2. **Ensamblado:** Traduce el fichero `.s` generado por nuestro compilador a código máquina.
  3. **Enlazado (Linking):** Combina el código máquina del programa con el del runtime para generar el ejecutable final (`.exe` en Windows o binario nativo en Unix).

  Instalación recomendada:

  - En **Linux/Mac**: Generalmente preinstalado o disponible vía repositorios (`apt install gcc`, `brew install gcc`). Además, debido a que MacOS utiliza procesadores ARM y nuestro compilador está pensado para Intel x86, se ha usado Docker Desktop para simular que el Mac sea un dispositivo Ubuntu de 32 bits.
  - En **Windows**: Es imprescindible instalar un entorno como **MinGW** o **TDM-GCC**. Es importante asegurarse de que la ruta a la carpeta `bin` de la instalación se añada al `PATH` del sistema para que el comando `gcc` sea accesible desde la terminal.

  Compatibilidad x64: Debido a que el diseño del back-end está pensado para una arquitectura de 32 bits, utilizando instrucciones específicas (como `PUSHL`) y una ocupación de 4 bytes para tipos de datos básicos (como el tipo *Enter*), se ha incorporado el flag `-m32` en el script de automatización `run.bat`. Esto garantiza que el código sea procesado correctamente por el ensamblador de GCC incluso con instalaciones de 64 bits.

6.2 Lenguaje

A continuación se muestra la estructura, sintaxis y reglas básicas para escribir programas compatibles con el compilador:

El lenguaje impone un orden estricto de declaración para simplificar la visibilidad de variables y funciones. Un archivo fuente (.txt) válido debe seguir este esquema secuencial:

1. **Zona de Declaraciones Globales:** Variables accesibles desde cualquier punto.
2. **Zona de Subprogramas:** Funciones y procedimientos.

3. **Bloque Principal (Main):** Punto de entrada del programa. **Debe estar siempre al final del archivo**.

```
// 1. Declaraciones Globales
int contador;
char letra;

// 2. Definici n de Funciones (Antes del main)
int sumar(int a, int b) {
    return a + b;
}

// 3. Main (Siempre abajo)
main() {
    contador = sumar(10, 20);
    imprimir(contador);
}
```

Listing 3: Esqueleto de un programa válido

6.2.1 Tipos de Datos y Operaciones

Elemento	Sintaxis / Descripción
Tipos Primitivos	`int` (Enteros 32-bit), `char` (ASCII o 'letra'), `bool` (Lógico).
Vectores (Arrays)	`tipo nombre[tamaño];` (El tamaño debe ser un literal entero).
Asignación	`var = valor;` o `vec[i] = valor;`
Aritmética	`+, -, *, /, %`
Lógica	`&& (AND), || (OR), ! (NOT), ==, !=, <, >, <=, >=`

Tabla 1: Resumen de tipos y operadores.

6.2.2 Entrada y Salida (I/O)

El lenguaje dispone de dos primitivas para la interacción con la consola:

- `leer(variable);` Lee un valor del teclado y lo guarda en la variable. Soporta enteros y caracteres.
- `imprimir(expresion);` Evalúa la expresión y muestra el resultado en pantalla.

6.2.3 Estructuras de Control

La sintaxis es similar a C/Java. Es obligatorio el uso de llaves `{ }` para delimitar los bloques de código, incluso si solo hay una sentencia.

```
// Condicional IF-ELSE
if (a > b) {
    imprimir(a);
} else {
    imprimir(b);
```

```
}

// Bucle WHILE
while (i < 10) {
    i = i + 1;
}

// Bucle FOR
for (i = 0; i < 10; i = i + 1) {
    imprimir(i);
}

// Selecci n SWITCH (Solo enteros o caracteres)
switch (opcion) {
    case 1:
        imprimir('A');
        break;
    default:
        imprimir('E');
}
```

Listing 4: Ejemplos de estructuras de control

6.3 Compilación y Ejecución

El proceso de construcción consta de cuatro fases automáticas:

1. Compilación de las clases Java (Front-End).
2. Generación del código ensamblador (`.s`) a partir del archivo fuente.
3. Ensamblado y enlazado con el runtime de C mediante GCC.
4. Ejecución del binario resultante.

   Antes de ejecutar los scripts, la terminal debe estar **situada en el directorio raíz del proyecto** (la carpeta donde se encuentran los archivos `run.sh` y `run.bat`).

6.3.1 En Linux / macOS / WSL

Hacer uso del script de shell. Es necesario dar permisos de ejecución la primera vez:

```
$ chmod +x run.sh
$ ./run.sh nombre_fichero.txt
```

6.3.2 En Windows (CMD / PowerShell)

Hacer uso del script Batch nativo. No requiere permisos especiales, pero asegura que `gcc` sea accesible desde la terminal:

```
> .\run.bat nombre_fichero.txt
```

Organización de Ficheros Si los programas fuente (.txt) se encuentran almacenados en subdirectorios (por ejemplo, dentro de una carpeta `tests/`), es necesario especificar la **ruta relativa** completa como argumento al invocar el script.

```
# Ejemplo: El fichero esta en la carpeta 'casos_prueba'

# En Linux / macOS
$ ./run.sh casos_prueba/calculadora.txt

# En Windows
> .\run.bat casos_prueba\calculadora.txt
```

Listing 5: Ejecución con rutas relativas

7 Casos de Prueba y Resultados

A continuación se detallan los casos de prueba diseñados para verificar el compilador. Se dividen en pruebas de funcionamiento correcto (para validar la generación de código) y pruebas de errores (para validar el sistema de detección y recuperación de errores).

7.1 Caso Correcto 1: Sistema de Tipos y Estructuras de Datos

Fichero: `testTipos.txt`

7.1.1 Descripción y Objetivos

Este caso de prueba es fundamental para verificar la fase de análisis semántico y la generación de código básico. Su objetivo es validar:

- **Gestión de Memoria:** Correcta reserva de espacio para variables globales de diferentes tamaños (`int` 4 bytes, `char/bool` 1 byte) y arrays contiguos.
- **Aritmética y Precedencia:** Verificación de la jerarquía de operadores matemáticos.
- **Lógica Booleana y Control de Flujo:** Comprobación de que las sentencias `if/else` generan los saltos (*jumps*) correctos en ensamblador basándose en expresiones lógicas.
- **Manipulación de Caracteres:** Validación del tratamiento de `char` como enteros pequeños (ASCII) y su aritmética.

7.1.2 Salida Esperada

Al ejecutar el programa compilado, la salida por consola debe ser exactamente:

```
-1-
25
5
-2-
1
0
11
0
-3-
```

```
ABZ
-4-
pabc
-5-
194
Y
```

Nota: Los valores de caracteres como 'a' dependen de la implementación ASCII estándar (97).

7.2 Caso Correcto 2: Control de Flujo y Funciones

Fichero: `testOperaciones.txt`

7.2.1 Descripción y Objetivos

Este caso de prueba evalúa la capacidad del compilador para gestionar estructuras de control complejas y la modularidad del código. Los objetivos principales son:

- **Gestión de Subprogramas:** Verificación de la llamada a funciones y procedimientos, incluyendo:
  - Paso de parámetros por valor.
  - Gestión de la pila (Stack Frame) para variables locales.
  - Retorno de valores (`return`) y su uso en expresiones.
- **Ámbito de Variables (Scope):** Distinción correcta entre variables globales (persistentes) y variables locales (temporales dentro del `main` o funciones).
- **Estructuras de Iteración:** Generación correcta de etiquetas y saltos condicionales para bucles `while` y `for`.
- **Sentencia Switch:** Evaluación de la selección múltiple y funcionamiento correcto de la instrucción `break` para evitar la ejecución en cascada.
- **Operador Módulo:** Verificación de la operación de resto de división (`%`) utilizada en la función `esPar`.

7.2.2 Salida Esperada

```
-1-
30
-2-
1
1
-3-
3
2
1
-4-
10
-5-
200
```

7.3 Caso Correcto 3: Expresiones Complejas y Operadores

Fichero: `testOperadores.txt`

7.3.1 Descripción y Objetivos

Este caso de prueba se centra en la capacidad del compilador para evaluar expresiones matemáticas y lógicas con la prioridad correcta. Los objetivos específicos son:

- **Precedencia de Operadores:** Verificar que el árbol sintáctico (AST) se construye respetando la jerarquía matemática estándar (multiplicación/división antes que suma/resta) y el uso de paréntesis para alterar dicha precedencia.
- **Operadores Relacionales:** Comprobación de todas las comparaciones numéricas (`<, >, ==, !=, <=, >=`) y su correcta traducción a saltos condicionales en el código ensamblador.
- **Álgebra Booleana:** Validación de los operadores lógicos `AND, OR, NOT` y su combinación con expresiones relacionales.
- **Análisis Léxico de Caracteres Especiales:** Verificación de que JFlex reconoce correctamente las secuencias de escape como el salto de línea (`\n`) o el tabulador (`\t`).
- **Acceso a Arrays:** Comprobación de la sintaxis de corchetes `[ ]` para la indexación de memoria.

7.3.2 Salida Esperada

Nota: La salida exacta de los caracteres especiales (`\n, \t`) dependerá de la implementación de la función `imprimir` en el runtime, pero se representan aquí como espacios en blanco.

```
-1-
13
7
30
3
1
14
20
-2-
1
0
1
0
1
0
-3-
0
1
1
0
-4-
200
    777
```

7.4 Caso Correcto 4: Evaluación de Expresiones Aritmético-Lógicas

Fichero: `testExpresionesAritmeticasLogicas.txt`

7.4.1 Descripción y Objetivos

Este caso de prueba profundiza en la evaluación de expresiones compuestas, verificando que el compilador resuelva correctamente la jerarquía de operaciones y la compatibilidad de tipos. Los objetivos son:

- **Jerarquía y Asociatividad:** Confirmar que la precedencia estándar (multiplicación/división sobre suma/resta) se respeta en expresiones sin paréntesis, y que los paréntesis fuerzan correctamente el cambio de orden de evaluación.
- **Aritmética de Tipos (Coerción):** Validar la interoperabilidad entre tipos enteros (`int`) y caracteres (`char`), asegurando que los caracteres puedan participar en operaciones aritméticas comportándose como sus valores ASCII.
- **Lógica Booleana Completa:** Verificación exhaustiva de las tablas de verdad para `AND`, `OR` y `NOT`.
- **Expresiones Mixtas:** Evaluación de condiciones que mezclan aritmética, comparaciones y lógica en una sola sentencia (ej. `if ((a * 2) == b)`), poniendo a prueba la gestión de registros temporales en la generación de código.

7.4.2 Salida Esperada

```
-1-
12
20
30
2
1
-2-
ABD
-3-
1
0
1
0
1
-4-
1
2
3
```

7.5 Caso Correcto 5: Entrada/Salida e Interactividad (Calculadora)

Fichero: `testCalculadora.txt`

7.5.1 Descripción y Objetivos

Este último caso crea una calculadora interactiva. Su objetivo principal es validar la comunicación entre el programa compilado y el sistema operativo (Standard Input/Output) a través de la librería de tiempo de ejecución (`runtime`). Aspectos clave evaluados:

- **Gestión de I/O (Input/Output):** Correcto funcionamiento de la instrucción `leer()` para capturar datos del usuario.
- **Manipulación de Arrays:** Escritura y lectura secuencial en memoria para simular cadenas de texto (construcción manual del string "Hola"), validando el cálculo de direcciones de memoria base + desplazamiento.
- **Flujo Completo:** Integración de bucles `for`, condicionales `switch` y operaciones aritméticas en un caso de uso real.

7.5.2 Ejemplo de Ejecución Interactiva

Dado que este test requiere intervención del usuario, a continuación se muestra una traza de ejecución simulando una suma. Los valores introducidos por el usuario se marcan entre corchetes **[ ]** para mayor claridad (en la consola real no aparecen corchetes).

Entrada del usuario: 10, +, 5

```
Hola
1: [10]
Op: [+]
2: [5]
=15
```

7.6 Caso Erróneo 1: Tipos y Semántica

Fichero: `testErroresSemanticos.txt`

7.6.1 Descripción y Objetivos

Este caso de prueba contiene código sintácticamente correcto (respeta la gramática) pero semánticamente inválido. Su objetivo es validar el **Analizador Semántico** y la **Tabla de Símbolos**. El compilador debe ser capaz de detectar y reportar:

- **Validación de Identificadores:** Detección de uso de variables no declaradas y detección de intentos de redeclaración de variables en el mismo ámbito.
- **Inferencia y Comprobación de Tipos:** Verificación estricta de tipos en asignaciones (ej. no asignar `bool` a `int`) y operaciones (ej. no sumar `int + bool`).
- **Gestión de Arrays:** Validación de que los índices de acceso sean de tipo entero y que no se intente indexar variables que no son arrays.
- **Sentencias de Control:** Comprobación de que las condiciones en `if` y `while` sean estrictamente de tipo booleano.
- **Llamadas a Subprogramas:** Verificación de la firma de las funciones (número y tipo de parámetros) y distinción entre variables y funciones llamables.

7.6.2 Salida Esperada (Log de Errores)

El compilador no debe generar código ensamblador, sino una lista de errores detallada indicando la línea y la causa del fallo semántico. Un ejemplo de salida válida sería:

```
[ERROR SEMANTICO] Linea 27, Columna 9: Variable no declarada: noExiste
[ERROR SEMANTICO] Linea 32, Columna 1: Variable duplicada: entero
[ERROR SEMANTICO] Linea 37, Columna 5: Tipos incompatibles en asignaci n.
[ERROR SEMANTICO] Linea 38, Columna 5: Tipos incompatibles en asignaci n.
[ERROR SEMANTICO] Linea 44, Columna 14: Operaci n aritmetica requiere operandos num ricos.
[ERROR SEMANTICO] Linea 47, Columna 9: Operacion logica requiere operandos booleanos.
[ERROR SEMANTICO] Linea 55, Columna 5: El indice del array debe ser entero.
[ERROR SEMANTICO] Linea 58, Columna 5: Tipos incompatibles en asignaci n a array.
[ERROR SEMANTICO] Linea 61, Columna 5: La variable 'entero' no es un array.
[ERROR SEMANTICO] Linea 67, Columna 0: La condici n del IF debe ser de tipo booleano.
[ERROR SEMANTICO] Linea 75, Columna 5: Tipo de argumento incorrecto en llamada a 'funcion' (arg 1).
[ERROR SEMANTICO] Linea 78, Columna 5: Numero de argumentos incorrecto en llamada a 'funcion'. Esperados: 1, Recibidos: 0
```

7.7 Caso Erróneo 2: Recuperación de Errores Sintácticos y Semánticos

Fichero: `testErroresMixtos.txt`

7.7.1 Descripción y Objetivos

Este caso de prueba somete al compilador a un escenario donde hay errores de forma (violaciones de la gramática) con errores de fondo (violaciones de tipos y lógica). Su objetivo es validar la resistencia a errores del proceso de compilación:

- **Recuperación en Cascada:** Verificar que un error sintáctico (ej. falta de punto y coma) no impide la detección de errores semánticos posteriores.
- **Validación Estructural:** Comprobar restricciones gramaticales fuertes, como la obligatoriedad de llaves `{ }` en las estructuras de control o la prohibición de sentencias ejecutables en el ámbito global.
- **Validación de L-Values:** Confirmar que instrucciones como `leer()` solo acepten variables modificables y no literales.

7.7.2 Salida Esperada (Log de Errores)

El compilador debe ser capaz de reportar todos los fallos listados a continuación:

```
[ERROR SINTACTICO] Linea 6, Columna 1: Token inesperado:
'globalX'. Se esperaba: [T_INT, T_CHAR, T_BOOL, T_VOID]
[ERROR SINTACTICO] Linea 45, Columna 5: Error de Sintaxis:
Probablemente falta un punto y coma (;) en la instruccion
   ANTERIOR a 'if'.
[ERROR SINTACTICO] Linea 46, Columna 9: Token inesperado:
'a'. Se esperaba: ['{']
[ERROR SINTACTICO] Linea 48, Columna 10: Token inesperado:
'10'. Se esperaba: [un identificador]
[ERROR SEMANTICO] Linea 21, Columna 5: Variable no declarada:
   noExiste
[ERROR SEMANTICO] Linea 25, Columna 1: Variable duplicada: a
[ERROR SEMANTICO] Linea 30, Columna 9: Operacion aritmetica
   requiere operandos num ricos.
[ERROR SEMANTICO] Linea 34, Columna 5: El indice del array debe
   ser entero.
```

7.7.3 Discusión de la Recuperación

Se observa, por ejemplo:

1. El error en la línea 6 (Asignación Global) es descartado y el parser se recupera para leer correctamente `int globalCorrecta`.
2. El error de falta de punto y coma en la línea 43 es detectado al encontrar la palabra reservada `if` en la línea siguiente. El parser asume el cierre de la instrucción y continúa analizando el condicional.
3. El error de `leer(10)` es sintáctico porque la gramática define `leer(lvalue)`, y un número literal no cumple con la producción de `lvalue`.

7.8 Caso Erróneo 3: Recuperación sintáctica de la Calculadora

Fichero: `testErrorCalculadora.txt`

7.8.1 Descripción y Objetivos

Este caso de prueba toma como base el programa de la calculadora que ya habíamos visto anteriormente, pero poniendo múltiples errores sintácticos (ausencia de delimitadores, operadores faltantes, estructuras mal formadas). El objetivo es validar la robustez del programa en frente de errores.

- **Recuperación en Declaraciones:** Verificar si el compilador puede seguir procesando variables tras un error en la definición anterior.
- **Recuperación en Estructuras de Control:** Comprobar si el parser se recupera de errores dentro de la cabecera de un bucle `for` o en los casos de un `switch`.
- **Gestión de Final de Archivo:** Validar el comportamiento ante un bloque `main` no cerrado correctamente.

7.8.2 Salida Real del Compilador

La ejecución muestra cómo el analizador identifica cada uno de los errores sin detener el proceso de análisis, proporcionando mensajes de error específicos y sugerencias de corrección cuando es posible (como el punto y coma faltante):

```
[ERROR SINTACTICO] Linea 8, Columna 1: Error de Sintaxis:
Probablemente falta un punto y coma (;) antes de int
[ERROR SINTACTICO] Linea 21, Columna 14: Token inesperado:
'o'. Se esperaba: ['=']
[ERROR SINTACTICO] Linea 28, Columna 23: Error de Sintaxis:
Probablemente falta un punto y coma (;) antes de i
[ERROR SINTACTICO] Linea 45, Columna 19: Token inesperado:
PUNTO_COMA. Se esperaba: [')', '[']
[ERROR SINTACTICO] Linea 59, Columna 13: Token inesperado:
resultado. Se esperaba: [':']
[ERROR SINTACTICO] Linea 0, Columna 0: Final de archivo
inesperado. Falta la llave de cierre '}' del main.
```

7.8.3 Análisis de Resultados

1. **Precisión:** A pesar de que los errores ocurren en contextos variados (declaración global, asignación, bucle, llamada a función y estructura switch), el parser localiza correctamente el token conflictivo.
2. **Recuperación:** El uso del símbolo `error` en la gramática permite descartar la instrucción defectuosa y resincronizarse con el siguiente punto y coma o llave de cierre, permitiendo detectar 6 errores distintos en una sola pasada.

7.9 Presentación de la práctica

Se ha realizado un breve video explicando brevemente el compilador y mostrando el funcionamiento del programa de la calculadora:

ENLACE VIDEO

8 Conclusión

El diseño e implementación de este compilador ha permitido aprender de manera práctica los conceptos teóricos fundamentales de la construcción de lenguajes de programación. El proyecto ha finalizado con la creación de una herramienta funcional capaz de transformar código fuente de alto nivel, con estructuras complejas, en ejecutables nativos.

A lo largo del desarrollo, se han visto diversos retos:

1. **Robustez en el Análisis:** La implementación de mecanismos de recuperación de errores en el parser (Modo Pánico) y la validación semántica estricta garantizan que el compilador no solo traduzca código correcto, sino que sea capaz de guiar al programador identificando fallos de sintaxis y tipos con precisión.
2. **Gestión de Memoria y Ámbitos:** La simulación de la pila de llamadas (*Stack Frame*) y la gestión de ámbitos mediante una Tabla de Símbolos basada en pilas de HashMaps

han sido importantes para soportar recursividad, ocultación de variables y paso de parámetros.

3. **Modularidad del Backend:** La decisión de utilizar un Código de Tres Direcciones (C3@) como paso intermedio ha dividido el análisis de la síntesis, facilitando la traducción a ensamblador. Asimismo, la integración con un *Runtime* en C para las operaciones de Entrada/Salida.

En conclusión, este proyecto no solo ha resultado en un compilador funcional capaz de procesar algoritmos, sino que también ha proporcionado un aprendizaje del funcionamiento interno del software a bajo nivel, desde la gestión de registros y la pila del procesador hasta la estructura gramatical del lenguaje.